Data-Driven Testing and Test Parameterisation

LESSON SPECIFICATIONS & ROADMAP

• Track: Playwright Automation with JavaScript + AI • Level: Intermediate
• Estimated Effort: 2.5–3 Hours • Prerequisites: Days 1–8 concepts (Locators, actions, web-first assertions, auto-waiting, h
• Runtime: Pure Modern JavaScript (.js) • Runner: @playwright/test

Master scalable test parameterisation in Playwright using pure JavaScript arrays and objects. Run parameterized suites across positive, negative, and boundary datasets without copy-pasting test logic.

Day 9 Learning Objectives

'Understand why data-driven testing is critical for regression coverage and maintainability.
'Structure test data using JavaScript arrays, objects, and arrays of objects.
'Iterate over datasets to dynamically generate individual, isolated Playwright test instances.
'Use template literals in test titles to ensure clear, descriptive test reports in CI.
'Differentiate between defining individual parameterized tests vs looping assertions inside a single test.
'Manage test data effectively: separating data from test logic, handling valid/invalid partitions, and generating unique records.
'Avoid combinatorial explosion and maintain focused, high-value test suites.

In software testing, the same business workflow (such as logging in, filling a customer registration form, or calculating shipping taxes) must often be verified against dozens of distinct input variations.

The Problem with Duplicated Test Logic
Without data-driven testing, junior engineers copy-paste the exact same 15-line test script 10 times, altering only the username or password. This leads to:
1. **Maintenance Nightmare: If a button locator or URL changes, you must edit 10 different test files.**
2. **Inconsistent Assertions: Slight typos creep into different copies of the test.**
3. **Bloated Test Suites: Repositories become filled with thousands of lines of boilerplate.**

The Data-Driven Approach
Data-driven testing separates the **test logic (the sequence of actions and assertions) from the **test data** (the inputs and expected results). A single parameterized test template is executed repeatedly against an array of datasets.**

Avoiding Combinatorial Explosion
A common mistake is testing every conceivable combination of 5 dropdowns with 10 options each (100,000 combinations!). Instead:
- Test the core happy path (valid combination).
- Apply **Equivalence Partitioning (EP) to test one representative from each valid and invalid class.**
- Apply **Boundary Value Analysis (BVA) to test minimum, maximum, and boundary transitions.**

Copy-Paste Anti-Pattern vs Data-Driven Pattern (Comparing redundant duplicate tests with a clean parameterized array.)

```
// BAD: 3 identical tests copy-pasted
test('login with user A', async ({ page }) => { ... });
test('login with user B', async ({ page }) => { ... });
test('login with user C', async ({ page }) => { ... });

// GOOD: Single parameterized loop
const testUsers = [
  { role: 'Admin', user: 'admin@apex.io', expectedHeader: 'Admin Console' },
  { role: 'Student', user: 'student@apex.io', expectedHeader: 'My Learning' },
  { role: 'Instructor', user: 'tutor@apex.io', expectedHeader: 'Course Builder' },
];
```

Because Playwright tests are executed by Node.js, you have the full power of native JavaScript data structures.

Arrays of Objects: The Gold Standard
The cleanest and most expressive format for test datasets is an **Array of Objects:**

```javascript
const registrationScenarios = [
  {
    testName: 'TC-1: Valid Standard Registration',
    fullName: 'Priya Sharma',
```

```
    email: 'priya.sharma@example.com',
    password: 'ValidPassword123!',
    expectedSuccess: true,
    expectedMessage: 'Welcome to RootAccess!',
  },
  {
    testName: 'TC-2: Missing Full Name Field',
    fullName: '',
    email: 'priya.sharma@example.com',
    password: 'ValidPassword123!',
    expectedSuccess: false,
    expectedError: 'Full name is required',
  },
  {
    testName: 'TC-3: Malformed Email Address (no @ symbol)',
    fullName: 'Priya Sharma',
    email: 'priyasharma.example.com',
    password: 'ValidPassword123!',
    expectedSuccess: false,
    expectedError: 'Please enter a valid email address',
  },
];
```

Key Design Rules for Test Datasets

1. **Include a Descriptive testName / title Property: This will be passed to `test()` so your test runner output is crystal clear.**

2. **Explicit Expected Outcomes: Always include the expected error message or expected success indicator directly in each object.**

3. **Type Safety & Consistency: Keep all objects in the dataset array conforming to the exact same schema.**

Structured Parameterisation Schema (A clean object schema for testing discount coupon codes.)

```
const couponScenarios = [
  { code: 'SAVE20', cartTotal: 100, expectedDiscount: 20, isValid: true },
  { code: 'SAVE50', cartTotal: 100, expectedDiscount: 50, isValid: true },
  { code: 'EXPIRED', cartTotal: 100, expectedDiscount: 0, isValid: false, error: 'Coupon has expired' },
  { code: 'INVALID', cartTotal: 100, expectedDiscount: 0, isValid: false, error: 'Promo code not recognized' },
];
```

In Playwright, you generate multiple individual tests by wrapping the `test()` call inside a JavaScript `for...of` loop or `forEach()` method.

CRITICAL: Individual Tests vs Looping Inside One Test

```javascript
// 'L ANTI-PATTERN: Looping assertions inside a SINGLE test
test('validate registration scenarios', async ({ page }) => {
  for (const scenario of scenarios) {
    // If scenario #2 fails, the entire test terminates immediately!
    // Scenarios #3 to #10 NEVER EXECUTE!
    // The report only shows "1 failed test", masking whether other cases pass.
  }
});

// '  BEST PRACTICE: Defining INDIVIDUAL tests from the dataset
for (const scenario of scenarios) {
  test(`Registration: ${scenario.testName}`, async ({ page }) => {
    // Each scenario runs in complete isolation with its own BrowserContext and Page!
    // If scenario #2 fails, scenarios #1 and #3 still run and pass!
    // Reports show exactly which dataset failed.
    await page.goto('/register');
    // ...
  });
}
```

Dynamic Test Titles with Template Literals
Always use ES6 template literals (backticks ``) to embed scenario details into the test name:
`test(`Registration Validation - ${scenario.field}: ${scenario.description}`, async ({ page }) => { ... });`

Ensuring Test Independence
Each iteration of the loop generates a distinct Playwright test instance. Playwright automatically provides a freshly isolated `page` and `context` for each test, ensuring zero state leakage between datasets!

Clean Parameterized Playwright Test Suite (A parameterized test file verifying form input validations.)

```
const { test, expect } = require('@playwright/test');

const loginCases = [
  { name: 'Locked Account', user: 'locked@apex.io', pass: 'Pass123!', error: 'Your account is locked' },
  { name: 'Invalid Password', user: 'valid@apex.io', pass: 'WrongPass', error: 'Invalid email or password' },
  { name: 'Nonexistent User', user: 'nobody@apex.io', pass: 'Pass123!', error: 'Invalid email or password' },
];

for (const data of loginCases) {
  test(`Login Failure: ${data.name}`, async ({ page }) => {
    await page.goto('/login');
    await page.getByLabel('Email').fill(data.user);
    await page.getByLabel('Password').fill(data.pass);
    await page.getByRole('button', { name: 'Sign In' }).click();

    await expect(page.getByRole('alert')).toHaveText(data.error);
  });
}
```

Managing test data effectively is the difference between a resilient automation suite and an unstable, high-maintenance burden.

1. Separating Data into Dedicated Files
Keep large datasets out of your `.spec.js` test files. Place them in a `test-data/` or `fixtures/` directory:

```text
my-project/
% % %  test-data/
%    % % %  registrationData.json
%    % % %  searchQueries.js
% % %  tests/
   % % %  registration.spec.js
```

2. Generating Unique Data for Database Records
When testing registration or entity creation, static emails like `test@example.com` will fail on the second run due to "Email already exists" duplicate constraints!
Always generate dynamic, collision-free identifiers:

```javascript
const uniqueEmail = `user_${Date.now()}_${Math.floor(Math.random() * 1000)}@apexcorp.io`;
```

3. Avoiding Hardcoded Secrets
Never commit passwords, API tokens, or production credentials in test data files:
- Use environment variables: `process.env.TEST_USER_PASSWORD`.
- Keep credentials out of version control with `.gitignore`.
- Use synthetic dummy passwords (`DummyTestPass123!`) for local input validation tests.

Generating Collision-Free Registration Data (A helper function that builds unique user objects.)

```
function createUniqueUser(overrides = {}) {
  const timestamp = Date.now();
  return {
    fullName: `QA Tester ${timestamp}`,
    email: `qa_auto_${timestamp}@testdomain.io`,
    password: 'SecureTestPassword123!',
    ...overrides,
  };
}
```

requirements.

Assignment Starter Code (Day 9)

```
// tests/registration_data_driven.spec.js
const { test, expect } = require('@playwright/test');

// 1. Comprehensive Test Dataset: 8 Distinct Registration Scenarios
const registrationDatasets = [
  {
    id: 'TC-REG-01',
    description: 'Valid standard registration with all required fields',
    fullName: 'Ananya Deshmukh',
    email: `ananya_${Date.now()}@apexcorp.io`,
    phone: '+919876543210',
    password: 'ValidPassword@2026',
    expectedStatus: 'success',
    expectedMessage: 'Account created successfully! Welcome to RootAccess.',
  },
  {
    id: 'TC-REG-02',
    description: 'Negative: Missing required Full Name field',
    fullName: '',
    email: `ananya_${Date.now()}@apexcorp.io`,
    phone: '+919876543210',
```

DAY 9 INTERVIEW PREPARATION QUESTION BANK

Q1. What is data-driven testing, and how do you implement it in Playwright with pure JavaScript?

[Difficulty: Beginner] Evaluation Focus: Understanding of test parameterisation and ES6 iteration patterns.

Model Answer: Data-driven testing is the practice of separating test data from test logic. In Playwright with pure JavaScript, we define an array of test scenario objects containing inputs, flags, and expected results. We then iterate over the array using a for...of loop or forEach(), defining an individual test(`Scenario: ${data.name}`, async ({ page }) => { ... }) for each item. This provides isolated browser contexts, individual reporting, and prevents one failing dataset from aborting the rest.

Follow-Up Question: "Why should you avoid looping assertions inside a single test() block?"

Q2. How do you handle unique constraints (like unique email or username) in data-driven registration tests?

[Difficulty: Intermediate] Evaluation Focus: Practical test data management and collision avoidance.

Model Answer: For tests that write new records to a database, hardcoded static values will fail on subsequent runs due to duplicate key violations. We generate dynamic, collision-free identifiers using timestamps (e.g. `user_${Date.now()}@domain.io`) or UUIDs. For negative validation tests checking format rules (e.g. missing @ symbol or boundary length), static data is completely safe because no valid record is persisted.

Follow-Up Question: "When would you use API teardown after data-driven registration tests?"

Q3. What is combinatorial explosion, and how do you decide which datasets to automate?

[Difficulty: Intermediate] Evaluation Focus: Test optimization, Equivalence Partitioning, and Boundary Value Analysis.

Model Answer: Combinatorial explosion occurs when trying to test every possible combination of inputs, resulting in massive, unmaintainable test suites. We manage this by applying Equivalence Partitioning—testing one representative from each valid and invalid class—and Boundary Value Analysis for edge values (min, max, just below, just above). We prioritize high-risk, business-critical flows and common user mistakes over exhaustive combinations.

Follow-Up Question: "How does pairwise or orthogonal array testing help reduce test combinations?"

Q4. Where should test data be stored in a production Playwright automation project?

[Difficulty: Intermediate] Evaluation Focus: Framework directory layout and clean architecture.

Model Answer: Test data should be stored in dedicated JSON or JavaScript files within a `test-data/` or `fixtures/` directory, separate from test specs in `tests/`. For sensitive data like credentials, we use environment variables loaded via `.env` and `process.env`, never committing plaintext passwords to source control. Small scenario arrays directly relevant to a single feature can reside in dedicated test data modules imported into the spec.

Follow-Up Question: "What are the pros and cons of JSON files vs JavaScript (.js) files for test data?"

Q5. How do you parameterize tests across different environments (e.g., Staging vs Production)?

[Difficulty: Scenario-Based] Evaluation Focus: Environment configuration and baseURL parameterisation.

Model Answer: In Playwright, environment parameterisation is handled via `playwright.config.js` and environment variables. We use `process.env.ENV` to dynamically set `baseURL` (e.g., `https://staging.app.io` vs `https://prod.app.io`) and load environment-specific test accounts. The test specs themselves use relative URLs like `await page.goto("/register")`, allowing the exact same test suite to run against any environment seamlessly.

Follow-Up Question: "How do you prevent destructive tests from running against a production environment?"

1. What is the primary advantage of data-driven testing in modern test automation?
Key Concept: Data-driven testing decouples test logic from test inputs, enabling maintenance of the test workflow in one place while scaling test coverage across dozens of datasets.

2. How do you parameterize tests in Playwright using pure JavaScript?
Key Concept: In Playwright JavaScript, you iterate over an array of data objects using for...of or forEach, declaring an individual test() for each dataset.

3. What happens if you loop through assertions inside a single test() block and the 2nd dataset fails?
Key Concept: In a single test, an unhandled assertion failure throws an error that terminates the test immediately, leaving all subsequent iterations unexecuted.

4. Why should test titles use ES6 template literals (backticks) in data-driven suites?
Key Concept: Interpolating scenario attributes into the test title gives every generated test a unique, descriptive name in HTML reports, CLI summaries, and CI artifacts.

5. What technique prevents database duplicate-key errors when running registration tests repeatedly?
Key Concept: Collision-free dynamic emails using timestamps or UUIDs ensure each test run registers a brand new account that does not conflict with existing database records.

6. Which testing design technique helps automation engineers select high-value test data without combinatorial explosion?
Key Concept: Equivalence Partitioning and Boundary Value Analysis focus testing on representatives of valid/invalid classes and transition boundaries, maximizing defect detection with minimal tests.

7. Where should sensitive credentials (passwords, tokens) be stored in an automation framework?
Key Concept: Sensitive credentials must be stored in environment variables, excluded from version control via .gitignore, and accessed securely via process.env.

8. What is true about browser context isolation in Playwright data-driven tests?
Key Concept: Playwright Test provides clean test isolation by default: each individual test() receives an isolated BrowserContext with clean cookies, storage, and cache.

9. When should test data be stored in external JSON files rather than inline JavaScript arrays?
Key Concept: External JSON files are ideal for large, complex datasets, multi-file reusability, and data generated by APIs, mock servers, or test-data generators.

10. In a registration test suite with 8 parameterized datasets, how many test results will Playwright report?
Key Concept: Because test() is invoked 8 times in the outer loop, Playwright registers and reports 8 distinct test executions, each with its own timing, traces, and pass/fail outcome.